

# TripPlanner: aplicación web para la gestión y planificación de viajes personales

Juan García Moreno

## Directores

Maria Luque Rodríguez  
Antonio Araúzo Azofra

## Contenido

|                                               |          |
|-----------------------------------------------|----------|
| <b>1.- Introducción</b>                       | <b>1</b> |
| <b>2.- Objetivos</b>                          | <b>2</b> |
| <b>3.- Antecedentes</b>                       | <b>2</b> |
| <b>4.- Descripción del proyecto</b>           | <b>2</b> |
| 4.1.- Partes del proyecto                     | 2        |
| 4.2.- Fases del proyecto                      | 2        |
| <b>5.- Distribución temporal del proyecto</b> | <b>3</b> |
| <b>6.- Recursos</b>                           | <b>1</b> |
| 6.1.- Recursos humanos                        | 1        |
| 6.2.- Recursos hardware                       | 4        |
| 6.3.- Recursos software                       | 4        |
| <b>7.- Documentos de que consta el TFG</b>    | <b>4</b> |
| <b>Bibliografía</b>                           | <b>5</b> |

# 0.- Tipo de proyecto

Debido a la naturaleza del proyecto y los objetivos que se pretenden cumplir a partir de la materialización de la idea, este TFG podemos clasificarlo dentro del tipo Análisis y resolución de casos prácticos reales en el ámbito de la ingeniería.

## 1.- Introducción

La planificación de viajes implica la gestión de múltiples elementos como fechas, presupuestos, transporte, alojamiento, actividades y gastos, que habitualmente se encuentran dispersos en distintas herramientas. Esta fragmentación dificulta la organización y el seguimiento adecuado de la información asociada a un viaje.

La idea de desarrollar este proyecto surge a partir de una experiencia personal directa. Como estudiante Erasmus, la realización frecuente de viajes se convierte en una actividad habitual, lo que implica organizar desplazamientos, alojamientos, presupuestos y actividades en contextos cambiantes y, en muchos casos, compartidos con otras personas. Durante estos viajes, es común que aparezcan problemas de organización derivados de la falta de una herramienta centralizada que permita gestionar toda la información de forma clara y accesible.

Esta situación lleva a la necesidad de disponer de una aplicación que facilitará la planificación y el seguimiento de los viajes, permitiendo tener en un único lugar todos los datos relevantes y reduciendo los errores y la desorganización.

## 2.- Objetivos

El objetivo principal de este Trabajo Fin de Grado es diseñar e implementar una aplicación web que permita la gestión integral de viajes personales, facilitando la planificación, organización, seguimiento y registro de la información asociada a los viajes de un usuario de forma centralizada y accesible.

De manera más específica, el proyecto persigue los siguientes objetivos:

- Desarrollar una aplicación web que permita crear, editar y organizar viajes, incluyendo información relevante como fechas, destinos, transporte, alojamiento, presupuesto y actividades.
- Implementar un sistema de control de gastos que permita registrar gastos variables, realizar conversiones de moneda y comparar el gasto real con el presupuesto definido.

- Incorporar funcionalidades de planificación mediante checklists y planificación diaria del viaje, con el fin de mejorar la organización previa y durante el viaje.
- Permitir el registro de viajes completados, incluyendo la posibilidad de añadir notas, valoraciones y contenido multimedia, así como su visualización mediante mapas y líneas temporales.
- Diseñar una arquitectura cliente–servidor basada en una API REST y una Single Page Application, garantizando la escalabilidad y mantenibilidad del sistema.
- Desplegar la aplicación en un entorno real utilizando contenedores Docker sobre un servidor propio, asegurando la portabilidad y reproducibilidad del sistema.

Estos objetivos son concretos, alcanzables y realistas dentro del marco temporal del Trabajo Fin de Grado, y permiten evaluar de forma objetiva el grado de cumplimiento del proyecto al finalizar su desarrollo.

### 3.- Antecedentes

El desarrollo de aplicaciones web para la gestión de información personal y la planificación de actividades es un ámbito ampliamente tratado tanto en el entorno académico como profesional. Existen numerosas aplicaciones orientadas a la planificación de viajes que ofrecen funcionalidades parciales, como la reserva de vuelos, la gestión de itinerarios o el control de gastos. Sin embargo, muchas de estas soluciones se centran en aspectos concretos del viaje y no proporcionan una herramienta integrada que cubra de forma unificada la planificación, el seguimiento y el registro posterior del viaje.

Desde el punto de vista académico, este proyecto se apoya en diversos trabajos y prácticas realizadas a lo largo del grado, que han permitido adquirir los conocimientos necesarios para abordar el diseño y desarrollo de TripPlanner. En la asignatura de Ingeniería del Software se introdujeron los modelos de desarrollo de software, así como el uso de UML y metodologías ágiles como Scrum, sentando las bases para la correcta planificación y estructuración de un proyecto software, tal y como se describe en la literatura clásica de ingeniería del software [1][2].

En la asignatura de Diseño y Construcción de Software se profundizó en el modelado mediante UML y en el uso del Proceso Unificado Ágil (AUP), aplicando diagramas como los de casos de uso, actividades y clases. Estos elementos permiten representar de forma estructurada el comportamiento y la arquitectura del sistema, tal y como se recoge en las guías de modelado UML [5].

Por su parte, en Ingeniería Web se trabajó con arquitecturas de desarrollo basadas en sistemas Linux y servidores web, así como con el diseño de aplicaciones web modernas basadas en APIs REST. El uso de este estilo arquitectónico permite una separación clara entre cliente y servidor y favorece la escalabilidad del sistema, siguiendo los principios definidos por Fielding en su trabajo sobre arquitecturas REST [3], así como desarrollos posteriores sobre servicios web RESTful [9].

Asimismo, durante las prácticas de esta asignatura se utilizó GitHub como sistema de control de versiones, permitiendo la gestión colaborativa del código, el seguimiento de cambios y la integración continua. El uso de sistemas de control de versiones distribuidos es una práctica fundamental en el desarrollo de software moderno y resulta clave para garantizar la trazabilidad y mantenibilidad del proyecto [13].

Además, en la asignatura de Ingeniería de Requisitos se trabajó de forma práctica el análisis y especificación de requisitos del sistema, así como el uso de herramientas de prototipado para la validación temprana de soluciones. En particular, se utilizó Figma como herramienta de diseño de interfaces y prototipado, lo que permitió representar de forma visual las distintas pantallas del sistema y evaluar la experiencia de usuario antes de la fase de implementación. El uso de prototipos contribuye a reducir errores de diseño y a mejorar la comunicación de los requisitos, tal y como se recomienda en la literatura sobre ingeniería de requisitos y diseño centrado en el usuario [14]

Las prácticas realizadas en Programación Web permitieron adquirir conocimientos sobre el funcionamiento básico de la web, la gestión de servidores y puertos, el uso de lenguajes como JavaScript y Java, y la aplicación de patrones de diseño como la arquitectura en capas, el uso de objetos DAO y DTO y la separación de responsabilidades. Estos principios de diseño son ampliamente utilizados en aplicaciones empresariales y están documentados en la literatura sobre arquitectura software [7][8].

Asimismo, la asignatura de Programación Orientada a Objetos proporcionó una base sólida sobre los principios de la orientación a objetos, fundamentales para el diseño modular, reutilizable y mantenible del software. Conceptos como encapsulación, herencia y polimorfismo, junto con el uso de patrones de diseño, han sido claves en el desarrollo del sistema, siguiendo las buenas prácticas descritas por Gamma et al. [4].

Asimismo, en la asignatura de Modelo y Diseño Avanzado del Software se trabajaron tanto en teoría como en prácticas los principios de diseño orientados a la calidad del software, haciendo especial énfasis en los fundamentos de Clean Code y los principios SOLID. Estos conceptos resultan esenciales para la construcción de sistemas mantenibles, extensibles y fáciles de comprender, y han influido directamente en la estructura del código y en la separación de responsabilidades del sistema desarrollado. Las buenas prácticas aplicadas en este proyecto siguen las

recomendaciones ampliamente aceptadas en la literatura sobre diseño y calidad del software [11][12].

En el ámbito de las bases de datos, las prácticas realizadas permitieron aprender el diseño conceptual mediante diagramas entidad–relación y el uso del lenguaje SQL para la definición y consulta de datos. Estos conocimientos han sido aplicados directamente en el diseño y construcción del modelo de datos del sistema, siguiendo los principios clásicos de los sistemas de bases de datos relacionales [6].

Finalmente, en la asignatura de Programación y Administración de Sistemas se adquirieron competencias relacionadas con el uso de entornos Linux, estándares POSIX y herramientas de contenedorización como Docker. El uso de contenedores permite facilitar el despliegue y la portabilidad de aplicaciones, tal y como se describe en trabajos relacionados con la virtualización ligera y el despliegue continuo [10].

En conjunto, estos antecedentes académicos y técnicos han permitido identificar las limitaciones de soluciones existentes y justificar la realización de TripPlanner como un proyecto integrador. A diferencia de aplicaciones ya existentes, este proyecto adopta un enfoque formativo y modular, centrado en la aplicación práctica de los conocimientos adquiridos durante el grado y en el diseño completo de un sistema software desde el análisis hasta el despliegue final.

Por otra parte, me basé en las siguientes aplicaciones web y móvil. En el ámbito de la planificación y organización de viajes existen aplicaciones consolidadas como Triplt y Wanderlog. Triplt se centra principalmente en la gestión automática de itinerarios, permitiendo al usuario centralizar reservas de vuelos, alojamientos y transportes a partir de correos de confirmación, generando una visión estructurada del viaje. Wanderlog, por su parte, ofrece herramientas más visuales para la planificación de rutas, actividades y lugares de interés, incluyendo mapas y colaboración entre varios usuarios. Aunque ambas aplicaciones resultan útiles para la organización del itinerario, su enfoque está limitado a la fase de planificación y no abordan de forma completa otros aspectos como el control detallado del presupuesto, la gestión de tareas previas al viaje o el registro posterior de la experiencia.

En cuanto a la gestión económica de los viajes, destacan aplicaciones como TravelSpend y Tricount. TravelSpend permite registrar gastos durante el viaje, definir presupuestos, trabajar con múltiples monedas y analizar los gastos por categorías, facilitando el control financiero. Tricount está especialmente orientada al reparto de gastos entre varias personas, siendo habitual su uso en viajes en grupo. Sin embargo, estas herramientas se centran exclusivamente en el aspecto económico del viaje y no incluyen funcionalidades de planificación de itinerarios, gestión de actividades o documentación de viajes pasados.

TripPlanner se diferencia de estas soluciones existentes por su enfoque integral. Mientras que las aplicaciones analizadas se especializan en una única dimensión del viaje, ya sea la planificación del itinerario o el control de gastos, TripPlanner propone una plataforma unificada que cubre todas las fases del viaje: planificación previa, seguimiento durante el viaje y registro posterior. Además, el proyecto incorpora funcionalidades como checklists, planificación diaria, control de presupuesto, visualización de rutas y documentación del viaje, integradas dentro de una arquitectura diseñada específicamente con criterios de ingeniería del software.

| <b>Funcionalidad / Aplicación</b>          | <b>Triplt</b> | <b>Wanderlog</b> | <b>TravelSpend</b> | <b>Tricount</b> | <b>TripPlanner</b> |
|--------------------------------------------|---------------|------------------|--------------------|-----------------|--------------------|
| <b>Planificación de itinerarios</b>        | Sí            | Sí               | No                 | No              | Sí                 |
| <b>Organización por días y actividades</b> | Parcial       | Sí               | No                 | No              | Sí                 |
| <b>Gestión automática de reservas</b>      | Sí            | No               | No                 | No              | No                 |
| <b>Visualización de rutas y mapas</b>      | Limitada      | Sí               | No                 | No              | Sí                 |
| <b>Gestión de tareas (checklists)</b>      | No            | No               | No                 | No              | Sí                 |
| <b>Control de presupuesto del viaje</b>    | No            | Parcial          | Sí                 | Sí              | Sí                 |
| <b>Registro de gastos detallado</b>        | No            | No               | Sí                 | Sí              | Sí                 |
| <b>Soporte para múltiples monedas</b>      | No            | No               | Sí                 | Sí              | Sí                 |

|                                                   |    |         |    |    |    |
|---------------------------------------------------|----|---------|----|----|----|
| <b>Reparto de gastos entre usuarios</b>           | No | No      | No | Sí | No |
| <b>Registro de viajes pasados (diario, notas)</b> | No | No      | No | No | Sí |
| <b>Almacenamiento de fotografías/archivos</b>     | No | Parcial | No | No | Sí |

## 4.- Descripción del proyecto

El proyecto consiste en el diseño y desarrollo de una aplicación web orientada a la gestión integral de viajes personales, denominada TripPlanner. El sistema tiene como finalidad ofrecer una plataforma centralizada que permita al usuario planificar, organizar y registrar toda la información relacionada con sus viajes, abarcando las fases previa, durante y posterior al viaje.

En la fase de planificación, la aplicación permitirá crear viajes definiendo atributos como fechas, destinos, presupuesto, transporte y alojamiento, así como planificar el viaje por días mediante la asignación de actividades y lugares a visitar. De forma complementaria, se incorporará un sistema de listas de tareas o checklists que facilite la organización previa al viaje, permitiendo gestionar el estado y la prioridad de cada tarea.

Durante el desarrollo del viaje, el sistema permitirá registrar gastos variables asociados a distintas categorías, recalculando de manera automática el gasto total y comparándolo con el presupuesto establecido. Esta funcionalidad tiene como objetivo facilitar el control económico del viaje y ofrecer al usuario una visión clara de su situación financiera en tiempo real.

Una vez finalizado el viaje, la aplicación permitirá registrar información adicional asociada a los viajes completados, como diarios o notas personales, valoraciones, fotografías y otros archivos. Asimismo, se ofrecerán distintas formas de visualización de esta información, incluyendo vistas detalladas, representación cronológica del viaje y visualización de los lugares visitados sobre un mapa geográfico.

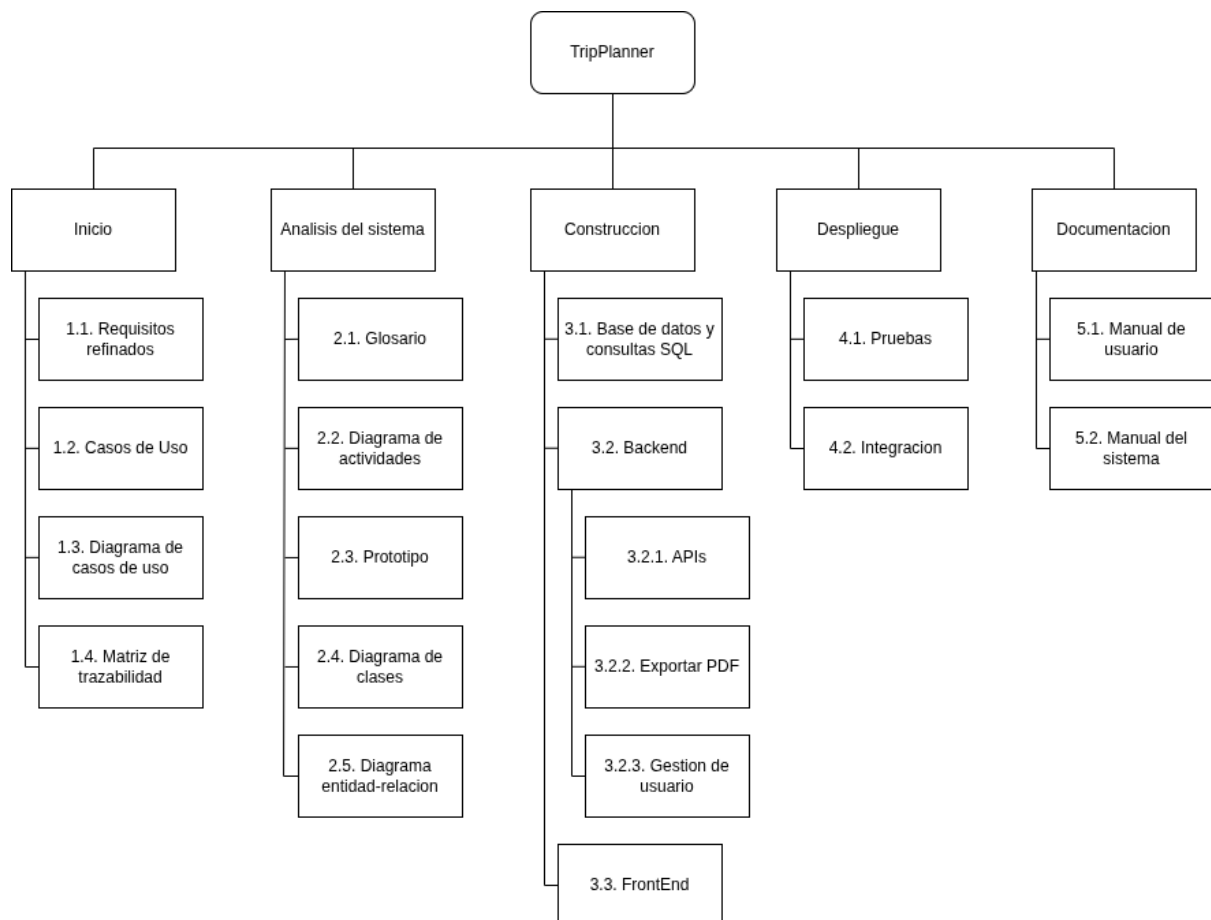
Desde el punto de vista técnico, el proyecto se basa en una arquitectura cliente-servidor desacoplada, estructurada en capas, que facilita la separación de responsabilidades y mejora la mantenibilidad del sistema. La comunicación entre el cliente y el servidor se realizará mediante una API REST, permitiendo una interacción eficiente y escalable. El sistema será desplegado en un entorno real

utilizando técnicas de contenedorización, lo que garantiza la portabilidad y consistencia del despliegue.

En conjunto, el proyecto pretende aplicar de forma práctica conocimientos adquiridos durante el grado en áreas como ingeniería del software, diseño de arquitecturas, bases de datos y desarrollo de aplicaciones web, integrándose en un sistema funcional orientado a resolver una necesidad real de los usuarios.

## 4.1.- Partes del proyecto

Para facilitar el desarrollo y la organización del proyecto, este se ha dividido en varias partes o bloques funcionales, siguiendo una estructura de descomposición del trabajo (Work Breakdown Structure, WBS). Esta división permite abordar el proyecto de forma incremental, mejorar la planificación y facilitar el seguimiento del progreso.





| <b>Código EDT</b> | <b>Paquete de trabajo</b> | <b>Descripción del trabajo</b>                                                                                                                                                                   |
|-------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1               | Requisitos refinados      | Revisión y depuración de los requisitos iniciales del sistema, eliminando ambigüedades, priorizando funcionalidades y definiendo con precisión el alcance funcional y no funcional del proyecto. |
| 1.2               | Casos de uso              | Identificación y definición de los casos de uso del sistema, describiendo las interacciones entre los usuarios y la aplicación, incluyendo actores, flujo principal y flujos alternativos.       |
| 1.3               | Diagrama de casos de uso  | Elaboración del diagrama UML de casos de uso, representando gráficamente los actores y las funcionalidades del sistema para ofrecer una visión global de su comportamiento.                      |
| 1.4               | Matriz de trazabilidad    | Definición de la relación entre requisitos y casos de uso, asegurando que todos los requisitos están cubiertos y facilitando el seguimiento de cambios durante el desarrollo.                    |
| 2.1               | Glosario                  | Creación de un glosario con los términos clave del dominio del sistema, con el objetivo de evitar ambigüedades y garantizar una comprensión común del proyecto.                                  |
| 2.2               | Diagrama de actividades   | Modelado de los principales flujos de trabajo del sistema mediante diagramas de actividades UML, describiendo el comportamiento dinámico de los procesos más relevantes.                         |
| 2.3               | Prototipo                 | Diseño de prototipos de interfaz de usuario para validar la estructura del sistema y la experiencia de usuario antes de la fase de implementación.                                               |
| 2.4               | Diagrama de clases        | Elaboración del diagrama de clases UML, definiendo las entidades del sistema, sus atributos, métodos y relaciones, sirviendo como base para la implementación orientada a objetos.               |

|       |                               |                                                                                                                                                                                         |
|-------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.5   | Diagrama entidad–relación     | Diseño conceptual de la base de datos mediante un diagrama entidad–relación, representando las entidades persistentes y las relaciones existentes entre ellas.                          |
| 3.1   | Base de datos y consultas SQL | Creación del esquema de la base de datos, definición de tablas, relaciones y restricciones, así como implementación de las consultas SQL necesarias para el funcionamiento del sistema. |
| 3.2   | Backend                       | Implementación de la lógica de negocio del sistema en el servidor, gestionando las operaciones relacionadas con viajes, gastos, usuarios y planificación.                               |
| 3.2.1 | APIs                          | Desarrollo de la API REST que permite la comunicación entre frontend y backend, definiendo endpoints, formatos de datos y reglas de acceso.                                             |
| 3.2.2 | Exportar PDF                  | Implementación de la funcionalidad que permite generar y exportar documentos PDF con información relevante de los viajes.                                                               |
| 3.2.3 | Gestión de usuario            | Desarrollo de las funcionalidades de registro, inicio de sesión, autenticación, autorización y gestión del perfil de usuario.                                                           |
| 3.3   | Frontend                      | Desarrollo de la interfaz de usuario de la aplicación web, implementando vistas, formularios y componentes necesarios para la interacción con el sistema.                               |
| 4.1   | Pruebas                       | Ejecución de pruebas funcionales para verificar el correcto funcionamiento del sistema y detectar posibles errores.                                                                     |
| 4.2   | Integración                   | Integración de frontend, backend y base de datos, asegurando el correcto funcionamiento conjunto del sistema en un entorno real.                                                        |
| 5.1   | Manual de usuario             | Elaboración de un manual dirigido a los usuarios finales, describiendo el uso de las principales funcionalidades de la aplicación.                                                      |

|     |                    |                                                                                                                 |
|-----|--------------------|-----------------------------------------------------------------------------------------------------------------|
| 5.2 | Manual del sistema | Redacción de la documentación técnica del sistema, incluyendo arquitectura, diseño, despliegue y mantenimiento. |
|-----|--------------------|-----------------------------------------------------------------------------------------------------------------|

## 4.2.- Fases del proyecto

El desarrollo del proyecto se ha organizado en una serie de fases orientadas a su planificación temporal. Estas fases no coinciden de forma estricta con la división funcional del WBS, sino que agrupan tareas relacionadas con el objetivo de permitir un desarrollo progresivo e incremental del sistema. De este modo, se prioriza la implementación temprana de funcionalidades básicas y se amplía el sistema de forma gradual hasta alcanzar la versión final.

### **Fase 1. Inicio y análisis de requisitos**

En esta fase inicial se realiza la extracción y refinamiento de los requisitos del sistema. Se identifican los objetivos del proyecto, el alcance funcional y las necesidades del usuario. Como resultado de esta fase se elaboran los casos de uso, los diagramas de casos de uso, la matriz de trazabilidad y el glosario, estableciendo una base sólida para el diseño y desarrollo posteriores.

### **Fase 2. Diseño del sistema**

Durante esta fase se define el diseño general del sistema a nivel lógico y estructural. Se elaboran los diagramas de actividades, el diagrama de clases y el diagrama entidad-relación de la base de datos. Asimismo, se desarrolla un prototipo de interfaz que permite validar la estructura de la aplicación y la experiencia de usuario antes de comenzar la implementación.

### **Fase 3. Implementación de la base de datos**

En esta fase se lleva a cabo la creación de la base de datos del sistema, definiendo su modelo, las relaciones entre entidades y las consultas SQL necesarias. Esta fase permite disponer de una persistencia de datos funcional que sirve de soporte al desarrollo del backend.

### **Fase 4. Desarrollo del backend**

Esta fase se centra en la implementación de la lógica de negocio del sistema. Se desarrollan los servicios del backend, incluyendo la gestión de usuarios, el control de autenticación, la creación de APIs y la implementación de las funcionalidades principales relacionadas con la gestión de viajes, gastos y planificación. El backend se desarrolla de forma incremental, permitiendo la validación progresiva de las funcionalidades.

### **Fase 5. Desarrollo del frontend**

En paralelo al backend, se desarrolla la interfaz de usuario de la aplicación web. En esta fase se implementan las distintas vistas del sistema y la comunicación con el backend, permitiendo al usuario interactuar con las funcionalidades disponibles. El

desarrollo del frontend se adapta al progreso del backend, integrando progresivamente las funcionalidades implementadas.

### **Fase 6. Integración y pruebas**

Una vez desarrollados los principales componentes del sistema, se realiza la integración completa entre frontend, backend y base de datos. En esta fase se llevan a cabo pruebas funcionales para verificar el correcto funcionamiento de las distintas funcionalidades y detectar posibles errores.

### **Fase 7. Despliegue y documentación**

En la fase final se realiza el despliegue del sistema en un entorno real y se elabora la documentación del proyecto. Se incluyen tanto el manual de usuario como la documentación del sistema, y se evalúa el grado de cumplimiento de los objetivos planteados al inicio del proyecto.

## **5.- Distribución temporal del proyecto**

El desarrollo del proyecto se ha planificado teniendo en cuenta una dedicación total aproximada de 300 horas, acorde a la carga de trabajo establecida para un Trabajo Fin de Grado del Grado en Ingeniería Informática en la EPSC. La planificación temporal se ha organizado en fases principales que permiten un desarrollo progresivo e incremental del sistema, facilitando el seguimiento del avance del proyecto y la consecución de los objetivos planteados.

Las fases principales del proyecto son las siguientes: análisis y diseño del sistema, implementación de la base de datos y backend, desarrollo del frontend, integración y pruebas, despliegue y documentación final. Cada una de estas fases agrupa un conjunto de tareas relacionadas, cuya duración ha sido estimada en función de su complejidad y del alcance del proyecto.

| <i>Tareas</i>                             | <i>Tiempo (h)</i> |
|-------------------------------------------|-------------------|
| Análisis y definición de requisitos       | 48                |
| + Extracción de requisitos                | 3                 |
| + Refinamiento de requisitos              | 2                 |
| + Casos de uso y diagrama de casos de uso | 15                |
| + Matriz de trazabilidad y glosario       | 1                 |
| + Prototipo                               | 5                 |
| + Diagramas de actividad                  | 20                |
| + Diagrama de clase                       | 2                 |
| Base de datos                             | 45                |

|                                               |            |
|-----------------------------------------------|------------|
| + Diagrama ER                                 | 5          |
| + Implementación de Base de Datos             | 30         |
| + Creación de esquema y relaciones            | 5          |
| + Consultas SQL y pruebas                     | 5          |
| <b>Desarrollo del Backend</b>                 | <b>75</b>  |
| + Desarrollo de API REST                      | 40         |
| + Usuarios y autenticación                    | 15         |
| + Lógica de negocio                           | 15         |
| + Tareas en segundo plano y generación de PDF | 5          |
| <b>Desarrollo del Frontend</b>                | <b>60</b>  |
| + Prototipo detallado                         | 5          |
| + Desarrollo de las vistas principales        | 35         |
| + Integración con API REST                    | 15         |
| + Visualización de mapas y rutas              | 5          |
| <b>Pruebas</b>                                | <b>20</b>  |
| + Pruebas funcionales                         | 10         |
| + Corrección de errores                       | 10         |
| <b>Desarrollo de contenedores</b>             | <b>2</b>   |
| <b>Configuración de entorno y despliegue</b>  | <b>10</b>  |
| <b>Documentación</b>                          | <b>40</b>  |
| <b>Total</b>                                  | <b>300</b> |

El cronograma del proyecto se ha planificado a lo largo de un periodo aproximado de 14 semanas, organizando las tareas en bloques temporales que permiten solapar determinadas actividades, como el desarrollo del backend y el frontend, optimizando así el tiempo disponible:

|                        | Marzo |   |   | Abril |   |   |   | Mayo |   |    |    | Junio |    |    |
|------------------------|-------|---|---|-------|---|---|---|------|---|----|----|-------|----|----|
| Semana                 | 1     | 2 | 3 | 4     | 5 | 6 | 7 | 8    | 9 | 10 | 11 | 12    | 13 | 14 |
| Análisis de requisitos |       |   |   |       |   |   |   |      |   |    |    |       |    |    |
| Base de Datos          |       |   |   |       |   |   |   |      |   |    |    |       |    |    |

|                       |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|-----------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
| Desarrollo Backend    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Desarrollo Frontend   |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Integración y pruebas |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Despliegue            |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Documentación         |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

## 6.- Recursos

### 6.1.- Recursos humanos

El proyecto será desarrollado por el estudiante Juan García Moreno, quien se encargará de todas las fases del mismo, incluyendo el análisis de requisitos, el diseño del sistema, la implementación, las pruebas, el despliegue y la documentación final.

La dirección académica del proyecto corre a cargo de María Luque Rodríguez y Antonio Araúzo Azofra, directores del Trabajo Fin de Grado, quienes supervisarán el desarrollo del proyecto, orientarán en la toma de decisiones técnicas y metodológicas y evaluarán el cumplimiento de los objetivos establecidos.

### 6.2.- Recursos hardware

Para el desarrollo del proyecto se utilizará el ordenador personal del estudiante, que servirá como entorno principal de desarrollo, pruebas y documentación. Este equipo cuenta con las prestaciones necesarias para el desarrollo de aplicaciones web modernas, incluyendo la ejecución de entornos de desarrollo, servidores locales y herramientas de contenedorización.

Adicionalmente, el sistema será desplegado en una Raspberry Pi 5 Model B, que actuará como servidor de producción. Este dispositivo permite simular un entorno real de despliegue con recursos limitados, aportando valor añadido al proyecto al trabajar con hardware específico. La Raspberry Pi 5 Model B se utilizará para ejecutar los distintos servicios del sistema mediante contenedores Docker, incluyendo el backend, el frontend y la base de datos.

## 6.3.- Recursos software

El desarrollo del proyecto se apoyará en diversas herramientas y tecnologías software que facilitan la implementación de la arquitectura propuesta y garantizan la viabilidad del sistema, siguiendo buenas prácticas ampliamente aceptadas en la ingeniería del software [1][2].

En cuanto al sistema operativo y entorno de desarrollo, se utilizará un entorno basado en Linux, junto con herramientas habituales de desarrollo de software y editores de código. El uso de sistemas basados en Unix es habitual en el desarrollo de aplicaciones web y servidores, ya que facilitan la automatización de tareas, la gestión de procesos y el despliegue de aplicaciones [8].

Para el backend se contempla el uso de frameworks y librerías orientadas al desarrollo de APIs REST, destacando FastAPI [15] para la implementación de la lógica de negocio, junto con un ORM para la gestión de la persistencia de datos en una base de datos relacional. El diseño de la API seguirá los principios de la arquitectura REST, favoreciendo una separación clara entre cliente y servidor y mejorando la escalabilidad del sistema, tal y como se describe en la literatura sobre servicios web RESTful [3][9]. Para la autenticación y autorización de usuarios se considera el uso de mecanismos basados en tokens, como JWT [16], alineados con prácticas habituales en aplicaciones web modernas. Asimismo, se emplearán herramientas para la gestión de tareas en segundo plano y la generación de documentos PDF como parte de la lógica del sistema.

En el frontend se utilizarán tecnologías orientadas al desarrollo de Single Page Applications, considerando frameworks basados en componentes y lenguajes con tipado estático para mejorar la mantenibilidad y robustez del código. Este enfoque está alineado con los principios de diseño modular y separación de responsabilidades descritos en patrones de arquitectura de aplicaciones empresariales [7][12]. También se hará uso de librerías de componentes visuales y herramientas para la visualización de mapas y rutas geográficas. Como prototipado previo se utilizará Figma, una herramienta de prototipado basada en la nube empleada para la definición temprana de interfaces de usuario y la validación de requisitos, siguiendo las recomendaciones de la ingeniería de requisitos [14].

Para la base de datos se empleará un sistema de gestión de bases de datos relacional, adecuado para un dominio con múltiples relaciones entre entidades y soporte de transacciones. El diseño y uso de este tipo de sistemas se fundamenta en los principios clásicos de los sistemas de bases de datos relacionales [6].

Finalmente, para el despliegue y la gestión del sistema se utilizarán herramientas de contenedorización y orquestación, que permiten desplegar de forma conjunta los distintos servicios del sistema, garantizando la portabilidad y consistencia del entorno, tal y como se describe en trabajos relacionados con Docker y la

virtualización ligera [10]. Asimismo, se empleará un sistema de control de versiones distribuido para la gestión del código fuente y el seguimiento de los cambios realizados durante el desarrollo, una práctica esencial en proyectos de software modernos [13].

## 7.- Documentos de que consta el TFG

El Trabajo de Fin de Grado (TFG) estará compuesto por tres documentos fundamentales, que ofrecen una visión completa del proyecto, su desarrollo, su funcionamiento y su implementación. Cada uno de estos documentos cumple una función específica, orientada a proporcionar la información necesaria para la correcta comprensión y evaluación del trabajo realizado:

1. **Memoria del TFG**

La memoria es el documento principal y más extenso, donde se describe de manera detallada todo el proceso de desarrollo del proyecto.

2. **Manual de Usuario**

El manual de usuario tiene como objetivo proporcionar una guía clara y accesible para los usuarios finales de la aplicación

3. **Manual de Código**

El manual de código está dirigido a los desarrolladores que puedan necesitar comprender o modificar el código fuente de la aplicación en el futuro

## Bibliografía

[1] Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

[2] Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

[3] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Recuperado el 9 de febrero de 2026, de

[https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding\\_dissertation.pdf](https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf)

[4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

[5] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The unified modeling language user guide* (2nd ed.). Addison-Wesley.

[6] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.



- [7] Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- [8] Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3rd ed.). Addison-Wesley.
- [9] Richardson, L., & Ruby, S. (2007). *RESTful web services*. O'Reilly Media.
- [10] Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
- [11] Martin, R. C. (2009). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- [12] Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- [13] Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress.  
Recuperado el 9 de febrero de 2026, de  
<https://git-scm.com/book/en/v2>
- [14] Sommerville, I., & Sawyer, P. (1997). *Requirements engineering: A good practice guide*. John Wiley & Sons.
- [15] Ramírez, S. (2023). *FastAPI documentation*.  
Recuperado el 9 de febrero de 2026, de  
<https://fastapi.tiangolo.com/>
- [16] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force.  
Recuperado el 9 de febrero de 2026, de  
<https://datatracker.ietf.org/doc/html/rfc7519>